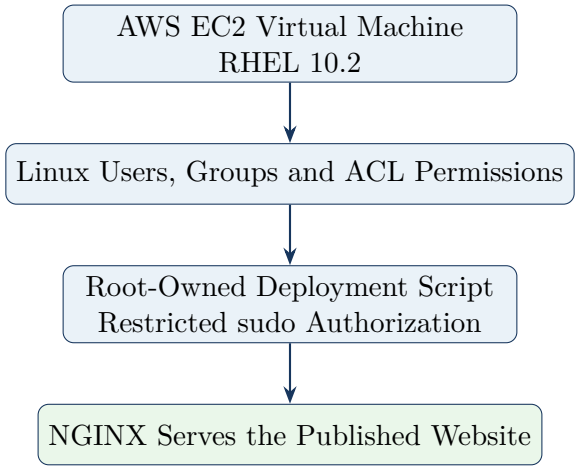
Linux Role-Based Access Control and Controlled Web Deployment

Practical Lab using AWS EC2, RHEL 10.2, ACLs, sudoers and NGINX



Purpose of this report

This report documents the implementation of role-based file access and controlled website deployment on a Linux server. It explains the configuration, the commands used, the reasons behind the permissions, and the verification results.

1 Objective

The objective was to configure and test a controlled web deployment environment on an Amazon EC2 instance running Red Hat Enterprise Linux (RHEL) 10.2.
The experiment focused on assigning permissions according to team responsibilities instead of granting every user unrestricted administrative access.
The main objectives were:

- Create Linux users and groups for different team roles.
- Configure ordinary file permissions and Access Control Lists (ACLs).
- Allow designated users to read, edit, deploy or clean up files according to their permissions.
- Install and configure the NGINX web server.
- Create a root-owned deployment script that DevOps members can execute through a specific sudoers rule.
- Verify the website deployment and test both permitted and restricted operations.

2 Prerequisites

2.1 Required Environment

The experiment used the following environment:

- Cloud platform: Amazon Web Services (AWS).
- Compute service: Amazon EC2.
- Operating system: Red Hat Enterprise Linux 10.2.
- Web server: NGINX 1.26.3.
- ACL utilities: `setfacl` and `getfacl`.
- Project location: `/srv/deployment-project/`.
- Deployment destination: `/usr/share/nginx/html/`.

The EC2 instance must be accessible to the administrator, and the administrator must have sufficient privileges to install packages, create users and groups, change permissions, configure sudoers, and manage system services.

2.2 Required Background Concepts

This experiment uses two important security concepts :

- **Role-based access control:** Permissions are assigned according to a user's responsibilities. For example, a developer may edit source code, while a different team member may only read it.
- **Least privilege:** Each user should receive only the permissions needed to perform their assigned tasks. A user who needs to deploy a webpage should not automatically receive unrestricted administrator access.

The lab also uses ACLs to provide different permissions to multiple groups on the same file. The detailed explanation of ACL entries, permission masks, directory permissions, and sudoers rules is provided alongside the implementation steps and in the glossary.

2.3 Team Access Plan

The following role assignments were configured:

Group	Members	Intended Responsibility
`aws-team`	Khagesh, Veenus	Read source HTML without modifying it.
`devops_team`	Manasi, Lahari	Edit the source and execute the approved deployment script.
`admin_group`	Veenus, Lahari	Manage entries in the designated cleanup directory.

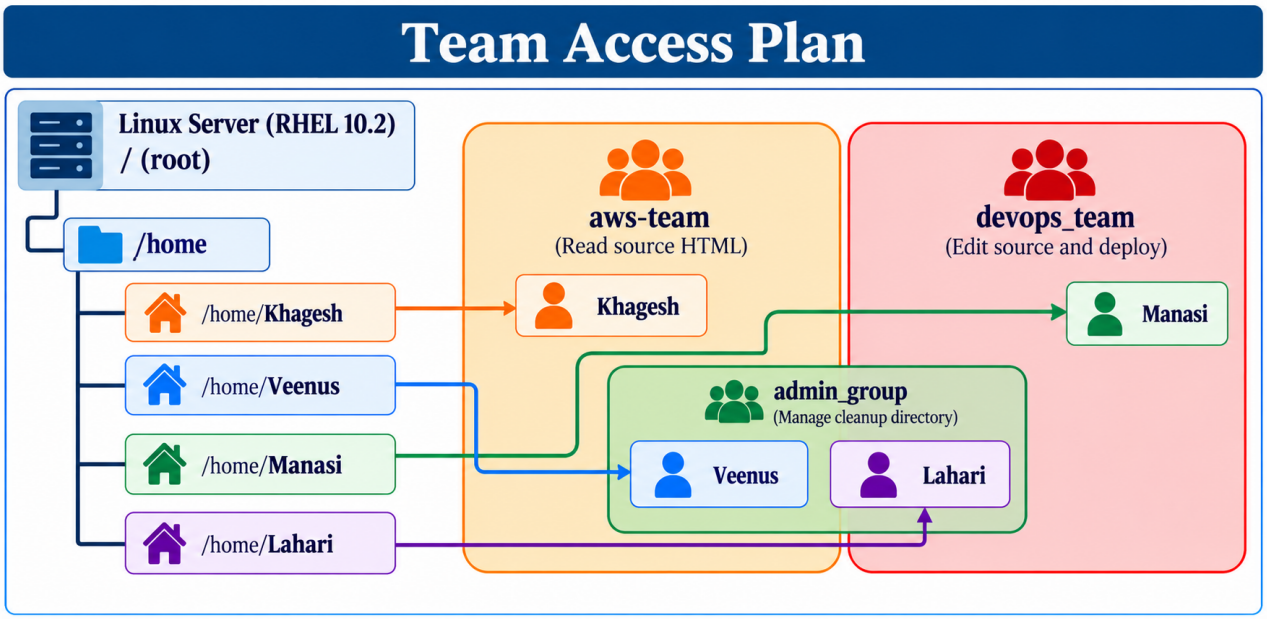


Figure 1: Team access plan showing Linux group memberships and overlapping roles

3 Description

The lab establishes a small website deployment workflow with separate locations for source files, prepared release files, and disposable cleanup files. The source HTML is stored in a protected project directory. DevOps members can modify the source, while members of the AWS team have read-only access to the source file. A deployment script validates the source file, copies it to a release location, and publishes it to the NGINX web root. The script is owned by root and cannot be modified by Manasi. DevOps members can execute the approved script through a dedicated sudoers configuration. The cleanup directory is controlled separately so that designated Admin group members can manage disposable entries without automatically gaining permission to remove the protected source file. NGINX serves the published HTML page over HTTP. The deployment was verified first with the original webpage and then with an updated paragraph edited by Manasi.

4 Topology

4.1 Architecture Diagram

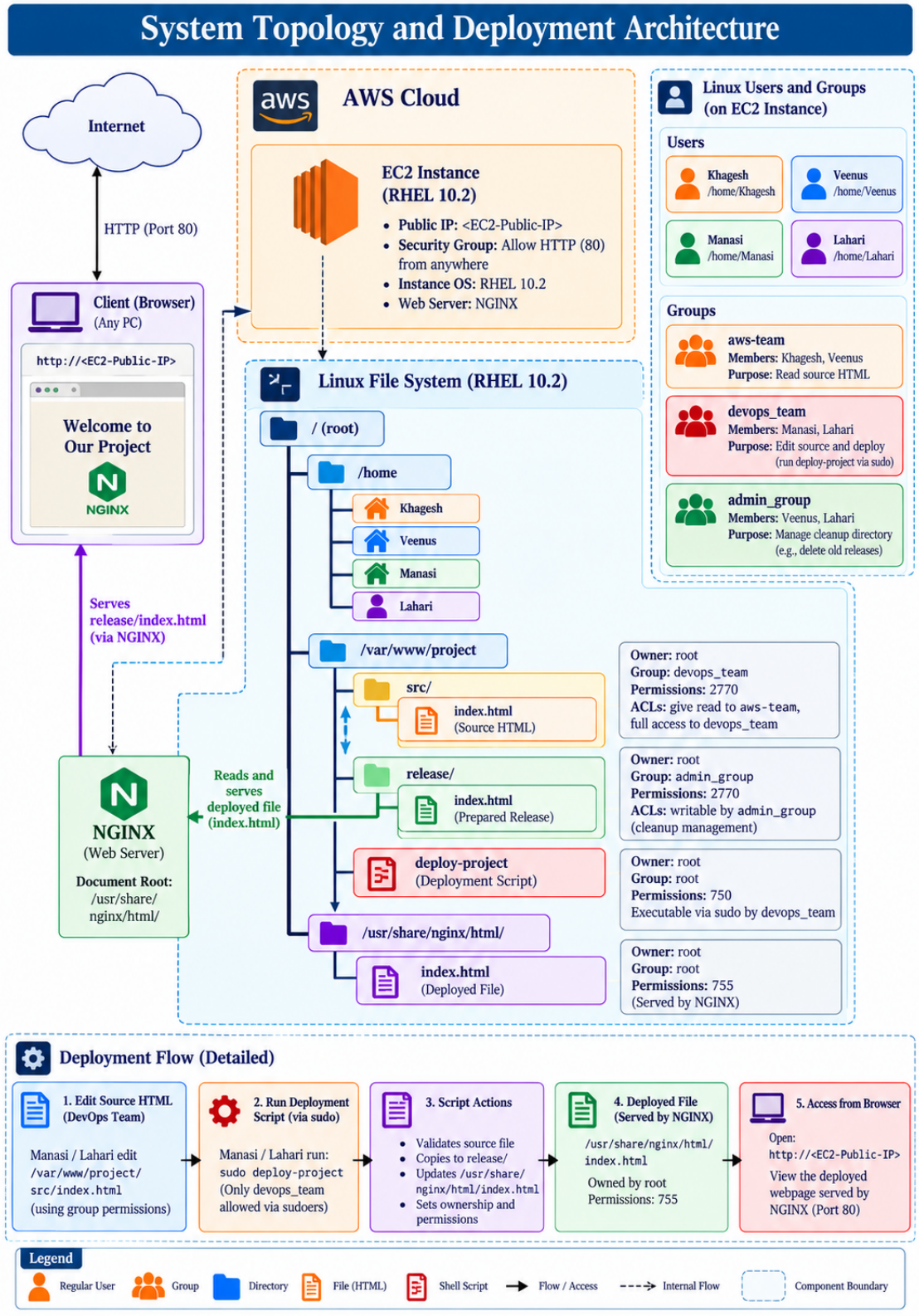


Figure 2: System topology and controlled deployment architecture

4.2 Project Directory Structure

```
/srv/deployment-project/
|-- src/
|   `-- index.html
|-- release/
|   `-- index.html
`-- cleanup/
    `-- test-artifact.txt
```

- `src/`: stores the source HTML file edited by DevOps.
- `release/`: stores the prepared copy produced by the deployment script.
- `cleanup/`: stores disposable files and directories used for Admin permission testing.
- `/usr/share/nginx/html/`: stores the published webpage served by NGINX.

The deployment script replaces the existing release and published files. Version history and rollback were outside the scope of this experiment.

5 Implementation Steps with Command Breakdowns

5.1 Step 1: Install Required Packages

The ACL utilities and NGINX were installed using:

```
sudo dnf install -y acl nginx
```

Command breakdown:

- `sudo` runs the command with administrative privileges.
- `dnf` is the package manager used by RHEL.
- `install` requests installation of the specified packages.
- `-y` automatically confirms package-manager prompts.
- `acl` provides the utilities needed to manage Access Control Lists.
- `nginx` installs the web server.

The installed utilities and NGINX version were checked:

```
command -v setfacl
command -v getfacl
nginx -v
```

Command breakdown:

- `command -v` identifies the executable path of a command.
- `setfacl` changes ACL entries.
- `getfacl` displays ACL entries.
- `nginx -v` displays the installed NGINX version.

The recorded environment showed the ACL utilities under `/usr/bin/` and NGINX version `1.26.3`.

5.2 Step 2: Start NGINX

NGINX was started and configured to start automatically after a reboot.

```
sudo systemctl enable --now nginx
systemctl is-active nginx
```

Command breakdown:

- `systemctl` manages system services.
- `enable` configures a service to start automatically during boot.
- `-now` also starts the service immediately.
- `is-active` checks whether the service is currently active.

The recorded service status was:

```
active
```

5.3 Step 3: Create Users and Groups

The three groups were created first, followed by the four user accounts.

```
sudo groupadd aws-team
sudo groupadd devops_team
sudo groupadd admin_group

for u in Khagesh Veenus Manasi Lahari; do
    sudo useradd -m -s /bin/bash "$u"
done

sudo usermod -aG aws-team Khagesh
sudo usermod -aG aws-team,admin_group Veenus
sudo usermod -aG devops_team Manasi
sudo usermod -aG devops_team,admin_group Lahari
```

Command breakdown:

- `groupadd` creates a new Linux group.
- `useradd -m` creates a user account and its home directory.
- `-s /bin/bash` selects Bash as the user's login shell.
- `for ... do ... done` repeats the enclosed commands for every username in the list.
- `$u` expands to the username currently stored in the loop variable.
- `usermod` modifies an existing user account.
- `-aG` appends supplementary group memberships without replacing the existing supplementary group list.

Group assignments were verified using:

```
getent group aws-team devops_team admin_group

id Khagesh
id Veenus
```

```
id Manasi
id Lahari
```

Command breakdown:

- `getent group` displays group records from the system's configured account databases.
- `id USER` displays the user's UID, primary group ID and group memberships.

The recorded group assignments were:

```
aws-team:x:1001:Khagesh,Veenus
devops_team:x:1002:Manasi,Lahari
admin_group:x:1003:Veenus,Lahari
```

Passwords were configured separately using `sudo passwd USER`. Passwords are intentionally excluded from this report.

5.4 Step 4: Create the Project Directories

The project directory structure and disposable test file were created using:

```
sudo mkdir -p /srv/deployment-project/src
sudo mkdir -p /srv/deployment-project/release
sudo mkdir -p /srv/deployment-project/cleanup
sudo touch /srv/deployment-project/cleanup/test-artifact.txt
```

Command breakdown:

- `mkdir` creates directories.
- `-p` creates missing parent directories when required.
- `touch` creates an empty file if it does not already exist.

The resulting structure separates source code, release output and disposable cleanup entries.

5.5 Step 5: Create the Source HTML

The original webpage contained a title, heading and introductory paragraph.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Team Deployment Project</title>
</head>
<body>
    <h1>Team Deployment Project</h1>
    <p>Deployed through a controlled Linux permissions workflow.</p>
</body>
</html>
```

The source was saved at:

```
/srv/deployment-project/src/index.html
```

HTML structures the content displayed by a web browser. The `title` element defines the browser-tab title, `h1` defines the main heading, and `p` defines the paragraph.

5.6 Step 6: Configure the Project Root

The project root was owned by root and given mode `0711`.

```
sudo chown root:root /srv/deployment-project
sudo chmod 0711 /srv/deployment-project
```

Command breakdown:

- `chown OWNER:GROUP PATH` sets the owner and owning group of a filesystem object.
- `chmod` changes the object's ordinary permission bits.
- Mode `0711` gives root full access and gives other users execute/traverse permission without read/list or write permission.

The restricted root directory allows access to a known path without granting ordinary users permission to list the project's top-level entries or create entries there.

5.7 Step 7: Configure the Source Directory

The source directory was assigned to root and the DevOps group. Its mode was set to `2770`.

```
sudo chown root:devops_team /srv/deployment-project/src
sudo chmod 2770 /srv/deployment-project/src

sudo setfacl -m g:aws-team:r-x \
  /srv/deployment-project/src

sudo setfacl -d -m g:aws-team:r-x,g:devops_team:rwx \
  /srv/deployment-project/src
```

Command breakdown:

- `chown root:devops_team` makes root the owner and DevOps the owning group.
- `chmod 2770` grants full directory permissions to the owner and group, none to others, and sets the setgid bit.
- `setfacl -m` adds or modifies access ACL entries.
- `g:aws-team:r-x` gives the named AWS group read and traverse permissions on the directory.
- `setfacl -d -m` configures a default ACL template for new entries in the directory.
- The default entries specify access for the AWS and DevOps groups when new entries are created, subject to normal file-creation permissions and ACL rules.

The setgid bit on a directory normally makes new files and subdirectories inherit the directory's owning group. Default ACLs provide a template for new entries; they do not automatically rewrite existing files.

5.8 Step 8: Configure the Release Directory

The release directory was owned by root with the DevOps group as its owning group.

```
sudo chown root:devops_team /srv/deployment-project/release
sudo chmod 2750 /srv/deployment-project/release
```

Command breakdown:

- `2750` sets the setgid bit.
- The owner permissions are `rwx`.
- The group permissions are `r-x`.
- Other users receive no permissions.

DevOps members can read and traverse the release directory but cannot directly create or delete entries there. The deployment script can write to it because the script runs as root.

5.9 Step 9: Configure the Cleanup Directory

The cleanup directory was configured for the Admin group.

```
sudo chown root:admin_group /srv/deployment-project/cleanup
sudo chmod 2770 /srv/deployment-project/cleanup

sudo chown root:admin_group \
  /srv/deployment-project/cleanup/test-artifact.txt

sudo chmod 0660 \
  /srv/deployment-project/cleanup/test-artifact.txt
```

Command breakdown:

- The directory owner is root and its owning group is `admin_group`.
- Mode `2770` gives root and the owning group full directory access and gives others no access.
- The setgid bit supports consistent group ownership for new entries.
- Mode `0660` gives the file owner and owning group read/write permissions.

The cleanup area was kept separate from the protected source directory so that deletion permissions could be tested independently.

5.10 Step 10: Configure the Source File ACL

The source file needed to be writable by the DevOps owning group and readable by the AWS group, without giving ordinary access to other users.
The named DevOps group entry was removed because DevOps was already the file's owning group.

```
sudo setfacl -x g:devops_team \
  /srv/deployment-project/src/index.html

sudo setfacl -m g::rw-,g:aws-team:r--,m::rw- \
  /srv/deployment-project/src/index.html

sudo getfacl /srv/deployment-project/src/index.html
```

Command breakdown:

- `setfacl -x` removes the specified named ACL entry.

- `g::rw-` sets read/write permissions for the owning group.
- `g:aws-team:r-` gives the named AWS group read-only access.
- `m::rw-` sets the ACL mask.
- `getfacl` displays the resulting ACL.

The final recorded ACL was:

```
# owner: root
# group: devops_team
user::rw-
group::rw-
group:aws-team:r--
mask::rw-
other::---
```

Interpretation:

- Root, the file owner, can read and write.
- The owning group, `devops_team`, can read and write.
- The named `aws-team` group can read the file.
- Other users have no permissions through the ordinary `other` entry.

The ACL mask limits the effective permissions of applicable group and named user/group entries. It does not independently grant permissions. Here, the AWS group's own entry is read-only, so the mask does not make it writable.

5.11 Step 11: Create the Deployment Script

The script was created at:

```
/usr/local/sbin/deploy-project
```

The complete script was:

```
#!/bin/bash
set -eu

SOURCE="/srv/deployment-project/src/index.html"
RELEASE_DIR="/srv/deployment-project/release"
RELEASE="$RELEASE_DIR/index.html"
WEBROOT="/usr/share/nginx/html"
DEST="$WEBROOT/index.html"
TEMP="$WEBROOT/index.html.new"

# Validate the source file
if [[ ! -s "$SOURCE" ]]; then
    echo "ERROR: Source HTML is missing or empty."
    exit 1
fi

# Prepare a controlled release copy
install -o root -g devops_team -m 0640 "$SOURCE" "$RELEASE"
```

```
# Publish the release
install -o root -g root -m 0644 "$RELEASE" "$TEMP"
mv -f "$TEMP" "$DEST"

# Restore the expected SELinux context
restorecon -v "$DEST"

echo "Deployment completed successfully."
```

Command and syntax breakdown:

- `#!/bin/bash` selects Bash as the script interpreter.
- `set -e` stops the script when a command fails.
- `set -u` treats the use of an unset variable as an error.
- The variables store the source, release, web-root, destination and temporary file paths.
- The `if` condition checks whether the source file is missing or empty. In Bash, `!` means "not", and `-s` checks that a file exists and has a non-zero size.
- `echo` prints a message, while `exit 1` stops the script with a non-zero status.
- `install -o root -g devops_team -m 0640` copies the source into the release location and sets its owner, group and permissions.
- `install -o root -g root -m 0644` creates the temporary published file owned by root and readable by other users.
- `mv -f` moves the temporary file to the final destination, replacing the existing file.
- `restorecon -v` restores the expected SELinux label on the destination file.
- The final `echo` prints the success message when the script reaches the end.

The `install` command in this script copies files and sets metadata; it is not the same as installing a software package.
The temporary file is created in the same directory as the final web file. Renaming a file on the same filesystem is normally atomic, helping prevent clients from seeing a partially copied destination.

5.12 Step 12: Protect the Deployment Script

The script was owned by root and given mode `0750`.

```
sudo chown root:root /usr/local/sbin/deploy-project
sudo chmod 0750 /usr/local/sbin/deploy-project
sudo bash -n /usr/local/sbin/deploy-project
sudo ls -l /usr/local/sbin/deploy-project
```

Command breakdown:

- `0750` gives root read/write/execute, the root owning group read/execute, and others no permissions.
- `bash -n` checks shell-script syntax without executing the script.
- `ls -l` displays file permissions, ownership and other file details.

The recorded permissions were:

```
-rwxr-x---. 1 root root 671 ... deploy-project
```

Manasi's write-access check returned:

```
sudo -u Manasi test -w /usr/local/sbin/deploy-project \
  && echo "UNEXPECTED: Manasi can modify the script" \
  || echo "PASS: Manasi cannot modify the script"
```

Command breakdown:

- `sudo -u Manasi` runs the check as Manasi.
- `test -w PATH` checks whether the current user has write access.
- `&&` executes the next command if the previous command succeeds.
- `||` executes the next command if the previous command fails.

The recorded output was:

```
PASS: Manasi cannot modify the script
```

Protecting the script matters because a user who could modify a script executed as root might be able to change what the administrator-level operation performs.

5.13 Step 13: Configure the sudoers Rule

The rule was added using the safe editor:

```
sudo visudo -f /etc/sudoers.d/deploy-project
```

The following rule was added to the file:

```
%devops_team ALL=(root) /usr/local/sbin/deploy-project
```

Rule breakdown:

- `%devops_team` applies the rule to members of the named group. The percent sign identifies a group.
- `ALL` specifies the hosts covered by the rule.
- `(root)` specifies that the command may run as root.
- The final path specifies the approved deployment script.

`visudo` edits and validates sudoers configuration. The recorded validation returned:

```
/etc/sudoers.d/deploy-project: parsed OK
```

The sudoers file was owned by root and had mode `0640`. The configured rule was also visible in the recorded sudo permission listings for Manasi and Lahari.

5.14 Step 14: Perform the Initial Deployment

The initial deployment was executed by the administrator.

```
sudo /usr/local/sbin/deploy-project
```

The script returned:

```
Deployment completed successfully.
```

The published file was owned by `root:root` with mode `0644`. The local HTTP header check returned `HTTP/1.1 200 OK`, confirming that the web server responded successfully.

6 Website Screenshots and Updated Deployment

6.1 SS1: Original Website

The original website displayed the heading "Team Deployment Project" and the paragraph describing the controlled Linux permissions workflow.



Figure 3: Original webpage before the source was edited by Manasi.

6.2 Step 15: Manasi Edits the Source HTML

Manasi changed the original paragraph using:

```
sudo -u Manasi sed -i \
's/Deployed through a controlled Linux permissions workflow./Updated by Manasi:
    DevOps deployment test successful./' \
/srv/deployment-project/src/index.html
```

Command breakdown:

- `sudo -u Manasi` executes the edit as Manasi rather than as root.
- `sed` processes text and can replace matching content.
- `-i` saves the replacement directly into the source file.
- The quoted expression specifies the original sentence and its replacement.

The change was verified as Manasi:

```
sudo -u Manasi grep -F "Updated by Manasi" \
  /srv/deployment-project/src/index.html
```

Command breakdown:

- `grep` searches text for a matching string.
- `-F` treats the search string as literal text.

The recorded output confirmed that the updated paragraph was present in the source file.

6.3 Step 16: Manasi Deploys the Update

Manasi executed the approved deployment script using:

```
sudo -u Manasi sudo /usr/local/sbin/deploy-project
```

The first `sudo -u Manasi` runs the command as Manasi. The inner `sudo` requests execution of the approved deployment script with root privileges, according to the sudoers configuration. Manasi authenticated when prompted.
The recorded output was:

```
Deployment completed successfully.
```

6.4 Step 17: Verify the Published Website

The published website was checked using:

```
curl -s http://localhost | grep -F "Updated by Manasi"
```

Command breakdown:

- `curl` requests the local website over HTTP.
- `-s` suppresses progress output.
- The pipe symbol `|` passes the returned HTML to the next command.
- `grep -F` prints the line containing the exact updated text.

The output contained:

```
<p>Updated by Manasi: DevOps deployment test successful.</p>
```

The release copy was verified using:

```
sudo ls -l /srv/deployment-project/release/index.html

sudo grep -F "Updated by Manasi" \
  /srv/deployment-project/release/index.html
```

The recorded release listing was:

```
-rw-r-----. 1 root devops_team 318 ... index.html
```

This confirmed that the release file existed, had the expected owner and group, and contained the updated paragraph.

6.5 SS2: Updated Website



Figure 4: Updated webpage after Manasi edited the source and deployed it.

Deployment result: PASS
Manasi edited the source, executed the approved deployment script, and published the updated page. The release copy contained the new paragraph, and NGINX returned that paragraph over HTTP.

7 Role-Based Testing and Verification

7.1 Test 1: AWS Team Can Read the Source

The source file was read as Khagesh:

```
sudo -u Khagesh cat /srv/deployment-project/src/index.html
```

Breakdown: `cat` prints the file contents. Running it with `sudo -u Khagesh` checks the read operation using Khagesh's permissions.
The command succeeded and displayed the HTML, including Manasi's updated paragraph.

7.2 Test 2: AWS Team Cannot Modify the Source

The write-access test was:

```
sudo -u Khagesh sh -c \
'echo "<!-- unauthorized change -->" >> /srv/deployment-project/src/index.html'
```

The recorded output was:

```
sh: line 1: /srv/deployment-project/src/index.html: Permission denied
```

Breakdown:

- `sh -c` asks a shell to execute the quoted command.
- `echo` produces the test text.
- » attempts to append that text to the source file.

The operation was denied because Khagesh had read access but not write access to the source file. This was the expected security result.

7.3 Test 3: Admin Group Can Delete Cleanup Entries

Veenus deleted the disposable test file:

```
sudo -u Veenus rm \
  /srv/deployment-project/cleanup/test-artifact.txt
```

The command completed successfully, and the cleanup directory listing became empty. An empty directory was then created for a separate removal test:

```
sudo mkdir /srv/deployment-project/cleanup/test-directory
sudo chown root:admin_group \
  /srv/deployment-project/cleanup/test-directory
sudo chmod 2770 \
  /srv/deployment-project/cleanup/test-directory
```

Veenus removed the empty directory:

```
sudo -u Veenus rmdir \
  /srv/deployment-project/cleanup/test-directory
```

Breakdown:

- `rm` removes a file.
- `rmdir` removes an empty directory.
- `chown` and `chmod` configure the test directory's ownership and permissions.

Both cleanup tests succeeded.

7.4 Test 4: Admin Member Cannot Delete the Protected Source

Veenus attempted to remove the source file:

```
sudo -u Veenus rm -f \
  /srv/deployment-project/src/index.html
```

The recorded output was:

```
rm: cannot remove '/srv/deployment-project/src/index.html': Permission denied
```

The deletion was denied as expected. Veenus belongs to `admin_group` but not to `devops_team`, so the DevOps source-directory permissions did not grant Veenus deletion access. The `-f` option suppresses interactive prompts; it does not bypass Linux permissions.

7.5 Test 5: Manasi Cannot Modify the Deployment Script

The test command was:

```
sudo -u Manasi test -w /usr/local/sbin/deploy-project \
  && echo "UNEXPECTED: Manasi can modify the script" \
  || echo "PASS: Manasi cannot modify the script"
```

The output was:

```
PASS: Manasi cannot modify the script
```

The test confirmed that Manasi could not write to the root-owned deployment script. This helps protect the integrity of the command that runs with root privileges.

8 Summary of Verification

Test	Expected Behaviour	Result
NGINX service status	Service active	PASS
Initial deployment	Script completes successfully	PASS
HTTP header check	HTTP 200 response	PASS
Manasi edits source	Write permitted	PASS
Manasi executes deployment script	Approved deployment permitted	PASS
Release copy contains update	Updated text present	PASS
NGINX serves updated HTML	Updated text returned	PASS
Khagesh reads source	Read permitted	PASS
Khagesh modifies source	Write denied	Expected denial
Veenus deletes cleanup file	Deletion permitted	PASS
Veenus deletes empty cleanup directory	Deletion permitted	PASS
Veenus deletes protected source	Deletion denied	Expected denial
Manasi modifies deployment script	Write denied	Expected denial

9 Glossary of Important Terms

Term	Meaning
AWS	Amazon Web Services, a cloud platform providing computing, storage, networking and other services.
EC2	Amazon Elastic Compute Cloud; a service that provides virtual machines called instances.
RHEL	Red Hat Enterprise Linux, the operating system used in this experiment.
NGINX	Web-server software that serves website files to clients.
HTTP	Hypertext Transfer Protocol, used by clients to request web resources.
HTML	HyperText Markup Language, used to structure webpage content.
Role-based access control	An approach in which permissions are assigned according to user roles and responsibilities.
Least privilege	The security principle of giving each user only the permissions necessary for assigned tasks.
User	An operating-system account with an identity and numeric user ID (UID).
Group	A collection of user accounts that can share permissions.

Root	The main Linux administrator account, normally with UID 0.
Owner	The user recorded as owning a file or directory.
Owning group	The group recorded as owning a filesystem object.
Read permission (`r`)	For files, permits reading contents. For directories, permits listing names.
Write permission (`w`)	For files, permits changing contents. For directories, can permit creating, removing or renaming entries when combined with the required directory permissions.
Execute permission (`x`)	For programs, permits execution. For directories, permits traversal and access to known entries subject to other checks.
UID	User ID, the numeric identity associated with a user account.
GID	Group ID, the numeric identity associated with a group.
Numeric permissions	Permission values calculated using read = 4, write = 2 and execute = 1.
Setgid bit	A special permission bit. On directories, it normally causes newly created children to inherit the directory's owning group.
ACL	Access Control List; a permission mechanism that can add named user or group entries beyond the basic owner, owning-group and other entries.
Access ACL	ACL entries that determine access to an existing file or directory.
Default ACL	A permission template on a directory used when new entries are created inside it. It does not automatically rewrite existing entries.
Named group ACL	An ACL entry that explicitly names a group, such as `group:aws-team:r-`.
`group::`	The ACL entry for the filesystem object's owning group.
`group:NAME:`	An ACL entry explicitly naming a group called NAME. This is distinct from the owning-group entry.
ACL mask	A limit on the effective permissions of applicable group and named user/group ACL entries. It does not grant permissions by itself.
Effective permissions	The permissions that actually apply after considering the relevant ACL entry and any applicable mask.
`setfacl`	A command used to add, modify or remove ACL entries.
`getfacl`	A command used to display ACL entries.
`chmod`	Changes ordinary filesystem permission bits.
`chown`	Changes the owner and/or owning group of a filesystem object.
`sudo`	Runs an allowed command with another user's privileges, commonly root.

sudoers	The configuration that determines which commands users may run through sudo and under which conditions.
`visudo`	A tool for editing and validating sudoers configuration.
Restricted sudo rule	A sudoers entry permitting a specific command rather than unrestricted administrator access. The actual scope also depends on the rest of the sudo policy.
Shell	A program that interprets commands and passes requests to the operating system. Bash is the shell used by the deployment script.
Variable	A named value used by a script. For example, `$SOURCE` expands to the path stored in the `SOURCE` variable.
Shebang	The initial `#!/path` line of a script, identifying the interpreter used to run it.
Exit status	A numeric result returned by a command. Zero conventionally means success; a non-zero value commonly indicates an error or other unsuccessful result.
`tee`	Reads input and writes it to a file, normally also printing the same input to the terminal.
Here-document	A shell feature, commonly written with `«EOF`, that supplies multiple lines of input to a command. The ending marker is chosen by the author.
Pipe (`|`)	Sends the output of one command to another command as input.
Output redirection (`>`)	Sends standard output to a destination, normally replacing an existing regular file.
Append redirection (`»`)	Appends standard output to a destination rather than replacing its existing contents.
Error redirection (`2>`)	Redirects standard error, conventionally file descriptor 2.
`/dev/null`	A special destination that discards data written to it.
`grep`	Searches text for matching patterns. The `-F` option treats the pattern as literal text.
`sed`	A text-processing tool used here to replace a sentence in the HTML source.
`curl`	A command-line tool that can request web resources using HTTP and other protocols.
`install`	A command that copies files and can set their ownership and permissions; in this script it is not installing a software package.
`mv`	Moves or renames a file. A same-filesystem rename is normally atomic.
`systemctl`	A command used to manage system services, including starting them and configuring automatic startup.
`bash -n`	Checks Bash script syntax without executing the script.
SELinux	Security-Enhanced Linux, an additional security mechanism that uses labels and policy rules to control access.

SELinux context	A security label associated with a filesystem object and used by SELinux policy.
`restorecon`	Restores the expected SELinux context on a file or directory.
Web root	The directory from which the web server serves files in a particular configuration.
Release copy	A prepared copy of the source file created before it is published to the web root.
Atomic publication	Replacing a file through an operation, such as a same-filesystem rename, that normally prevents clients from observing a partially copied destination file.
Permission denied	An operating-system response indicating that the requested operation was not permitted. In a security test, an expected denial can demonstrate that access controls are working.

10 Conclusion

The experiment successfully demonstrated controlled static website deployment on an AWS EC2 instance running RHEL 10.2.
NGINX served the original webpage and then the updated webpage after Manasi edited the source and executed the approved deployment script.
The permission tests demonstrated that:

- Khagesh could read the source file but could not modify it.
- Manasi could edit the source and deploy the updated page.
- Manasi could not modify the root-owned deployment script.
- Veenus could delete designated cleanup entries but could not delete the protected source file.
- The release copy and published website contained the updated paragraph.

The lab demonstrated how Linux users, groups, ordinary permissions, ACLs, directory permissions, restricted sudo authorization and SELinux work together to support controlled access and deployment.

Final outcome: PASS
The main learning objectives were achieved. The website was deployed successfully, the role-based access tests produced the expected results, and the report records the verified implementation. Release versioning and rollback were intentionally excluded from the experiment.